

GUÍA DE FUNCIONES

1. Construya la función `signo(x)`, la que debe tomar como parámetro un número y retornar 1, si este es positivo, -1, si es negativo o 0, si es cero.
2. Construya una función que, a partir de dos puntos representados como listas de la forma `[x, y]`, retorne como resultado la distancia entre estos puntos. Si el programador no ingresa el segundo punto, la función debe asumir que este es el punto (0,0) (representado por la lista `[0, 0]`).
3. Construya un programa que convierta valores de longitud/distancia entre una unidad y otra, por ejemplo, el programa solicita un valor de longitud, la unidad en la que está expresada y la unidad a la que se quiere convertir y el sistema realiza las conversiones. Considere para su solución que las unidades de distancia/longitud que debe considerar son:
 - Metro.
 - Kilómetro.
 - Centímetro.
 - Milímetro.
 - Milla.
 - Legua.
 - Pulgada.
 - Yarda.
 - Año Luz.

Además, considere que cada transformación debe ser realizada en una única función, sin embargo, algunas podrían tomar parámetros opcionales. Por ejemplo, puede definir la función `pulgada_a_m`, cuyos parámetros sean la cantidad de pulgadas a transformar y un parámetro opcional, que indique si se entrega el resultado en metros, kilómetros, milímetros o centímetros.

4. Construya una función que obtenga la raíz digital de un número entero. Esta es un número de 0 a 9 que se obtiene al sumar todos los dígitos del número. Si el resultado es mayor a 10, se repite la suma sobre este nuevo resultado hasta obtener un número entre 0 y 9, ambos inclusive.

Restricción: construya esta función de manera recursiva.

Por ejemplo, para el número 24.589:

$$\begin{aligned}24,589 &\rightarrow 2 + 4 + 5 + 8 + 9 \\&= 28 \\28 &\rightarrow 2 + 8 \\&= 10 \\10 &\rightarrow 1 + 0 \\&= 1\end{aligned}$$

por lo tanto, su raíz digital es 1.

5. Escriba una función cuyo resultado sea un booleano que indique si el número dado como argumento es un número fuerte o no. Un número fuerte es aquel que es igual a la suma de los factoriales de sus dígitos. Por ejemplo,

$$145 \rightarrow 1! + 4! + 5! = 1 + 24 + 120 = 145$$

Resuelva este problema utilizando el módulo `math`.

6. Construya una función que retorne si una lista de números entregada como argumento está ordenada de menor a mayor.
7. Construya una función que tome como parámetros una lista y un valor y entregue como resultado una copia de la lista donde se haya eliminado el valor.
8. Cree una función que entregue la intersección entre dos listas y otra que entregue la unión entre ambas.
9. Construya una función que cuente las vocales de un string sin utilizar métodos (como `count`) ni ciclos `while` ni `for`.
10. Construya una función que solo usando `pop`, `append`, `len` y bloques condicionales (`if-elif-else`), invierta una lista. Por ejemplo, si la lista de entrada es `[1, 2, 3, 4]`, el resultado es `[4, 3, 2, 1]`.
11. Escriba una función que encuentre el máximo común divisor entre dos números enteros utilizando el Algoritmo de Euclides:

- a) Sean $a, b \in \mathbb{Z}; a \geq b$.
- b) Si $b = 0$, el mínimo común divisor de a y b es a .
- c) En otro caso, el mínimo común divisor de a y b es el mínimo común divisor entre b y r , donde r es el resto de la división de a por b .

Note que este procedimiento es recursivo: refleje ese hecho en la función que escriba.

12. Una forma posible (si bien ineficiente) de determinar si un número entero positivo n es par o impar de manera recursiva:

- a)* n es par si es 0.
- b)* Si no es cero, n es par si $n - 1$ es impar.
- c)* n es impar si no es par.

Construya dos funciones, `es_par` y `es_impar` que implementen esta lógica recursiva.

13. Escriba una función que, sin utilizar `len` ni ciclos, encuentre cuántos caracteres tiene un string.